EXPERIMENT 4
Threads

OBJECTIVE:

- To understand and learn about threads and their implementation in programs

BACKGROUND:

Threads:

pthread_create(pthread_t* , NULL, void*, void*)

- First Parameter is pointer of thread ID it should be different for all threads.
- Second Parameter is used to change stack size of thread. Null means use default size.
- Third perimeter is address of function which we are going to use as thread.
- Forth parameter is argument to function.

pthread_join(i pthread_t , void**)

Pthread join is used in main program to wait for the end of a particular thread.

- First parameter is Thread ID of particular thread.
- Second Parameter is used to catch return value from thread.

Thread Library:

- POSIX Pthreads
- Two general stratergies for creating multiple threads.
  a) Asynchronous threading:
     - Parent and child threads run independently of each other
     - Typically little data sharing between threads
  b) Synchronous threading:
     - Parent thread waits for all of its children to terminate
     - Children threads run concurrently
     - Significant data sharing

Pthreads:

- **pthread.h**
- Each thread has a set of attributes, including stack size and scheduling information
- In a Pthreads program, separate threads begin execution in a specified function
- When a program begins
  - A single thread of control begins in main()
  - main() creates a second thread that begins control in the runner() function
  - Both threads share the global data

Example:

Write a program using **POSIX threads (pthread)** that creates two threads:

- The **first thread** should print even numbers from 0 to 10.
- The **second thread** should print odd numbers from 1 to 9.

Each thread should print one number per second. The main function should wait for both threads to finish before exiting.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h> // for sleep()
// Function for the first thread
void* evenNumbers(void* arg) {
   for (int i = 0; i <= 10; i += 2) {
      printf("Even Thread: %d\n", i);
      sleep(1);
   }
   return NULL;
}
// Function for the second thread
void* oddNumbers(void* arg) {
   for (int i = 1; i < 10; i += 2) {
      printf("Odd Thread: %d\n", i);
      sleep(1);
   }
   return NULL;
}
int main() {
```

```
    pthread_t thread1, thread2;

    // Create the threads
    pthread_create(&thread1, NULL, evenNumbers, NULL);
    pthread_create(&thread2, NULL, oddNumbers, NULL);

    // Wait for both threads to complete
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    printf("Main Thread: All threads finished.\n");
    return 0;
}
```

Question 1:

Write a C++ program using **POSIX threads (pthread)** that performs the following:

- Create **three threads**:
    1. The **first thread** should print "Thread A is running" **5 times**.
    2. The **second thread** should print "Thread B is running" **5 times**.
    3. The **third thread** should print "Thread C is running" **5 times**.

Each thread should sleep for **1 second** between each print.

Make sure the main() function waits for all three threads to complete before exiting.
